

AI-Powered QA Automation, Documentation & Software Engineering Assessment

Internship Hiring Assessment | Tactive

Duration: 1 week

1. Assessment Overview

What this is. A one-week, build-it-yourself assessment. You will design and build a small working web application, then use AI tooling to generate and run its test automation, make a feature change through that AI loop, and produce a professional documentation set — finishing with a short recorded demo.

Why it looks like this. We are not testing whether you can write code from memory. We are testing whether you can *direct* AI to build, test and correct real software. **Using AI is expected and encouraged.** What we assess is the quality of the system you end up with, the judgement behind your choices, and your ability to explain and demonstrate it live.

What we are looking for. Engineering judgement under an open brief, the ability to close a build → test → fix loop rather than just generate code, care about security and correctness, clear written and spoken communication, and honesty about what did and did not work.

A well-reasoned partial solution with an honest account of where it broke will score higher than a polished-looking submission that does not actually run. Do not fake results.

2. What You Will Build

Stage 1 — Pick a scenario and build a web app

Choose your own scenario. Anything real enough to have rules and edge cases — a library issue/return system, a canteen ordering system, a leave-approval workflow, a small inventory or invoicing tool. Pick **one specific functionality** within it and build that properly rather than building five features shallowly.

Any technology stack you like. It must run, and someone else must be able to start it from your repository using your README.

Stage 2 — AI-generated test automation

Use an AI tool to generate the test scripts for your chosen functionality, then actually **run** them and capture the results. Cover the normal path, the edge cases and the invalid inputs.

Important: a test that always passes proves nothing. Your suite must be able to fail. Show us at least one run where you deliberately break the application and the tests catch it (a red run).

Stage 3 — The AI change loop (*the core of this assessment*)

This is the part we care about most. Give the AI a **new feature request** for your application, and have it:

1. implement the change in your codebase,
2. run your existing test suite against it,
3. detect what broke, and
4. correct itself until the suite passes again.

Then capture the evidence: prompts given, what the AI changed, which tests failed, what it did about it, and how many attempts it took. Where the loop failed or you had to step in manually, say so.

Stage 4 — Documentation set

Produce, with AI assistance:

- **Architecture document** — components, data flow, technology choices and *why*.
- **Design document** — data model, key flows, API or interface design, error handling.
- **User guide** — how a non-technical person uses the feature you built.

3. Deliverables

#	Deliverable	Format
1	Source code repository (app + tests), with a README that lets us run it	Git repo or zip
2	Test suite plus captured run output, including one deliberate red run	Reports/logs/screens
3	AI change-loop evidence log — prompts, changes, failures, corrections, attempts	Markdown or PDF
4	Architecture, Design and User Guide documents	PDF or Markdown
5	Presentation deck (AI-generated is fine)	PDF or PPT or HTML
6	Video — 5 minutes total: 2 min problem → approach → solution, 3 min live demo	MP4 or public link

4. Ground Rules

- **AI tools are allowed and expected** — Claude, ChatGPT, Copilot, Cursor, or any coding agent.
- **Name your tools.** List every AI tool you used and what you used it for.

- **Your work must run.** We will start your application and run your tests from your repository.
- **Be prepared to defend it.** Shortlisted candidates will be asked to walk through their code and explain design decisions live. Submitting work you cannot explain is the fastest way to be ruled out.
- **Individual submission.** Discuss approaches with peers if you like; the build must be yours.

Suggested starting points (not prescriptive): web — React, Next.js, Django, Flask, Spring Boot, Node/Express. Testing — Playwright, Selenium, Cypress, pytest, JUnit, Jest. CI — GitHub Actions. Free tiers of every one of these are sufficient; do not spend money on this assessment.

5. How You Will Be Evaluated

Criterion	Weight	What earns marks
Completeness of the solution	25%	It runs; the loop in Stage 3 genuinely closes end to end
Complexity and ambition	20%	A real problem with real rules, not a to-do list
Innovation in AI orchestration	20%	How intelligently you directed the AI; automation of the loop itself
Security	15%	Input validation, auth where relevant, no secrets committed to the repo
Documentation and communication	15%	Clear docs, clear deck, a confident and well-paced demo
Handling of failure	5%	Honest diagnosis of what broke and what you would do next

6. Timeline and Submission

- **Time allowed:** one week from the date this brief is issued.
- **Submit:** all six deliverables above, in one folder or repository, to the address on your invite.
- **Questions:** ask early rather than guessing — clarifying an ambiguous brief is part of engineering.

If you do not finish: submit anyway. Include a short note covering what you completed, what you did not, where it broke, and what you would do with another week. That note is read carefully.